PHẦN 1: MÔ TẢ BÀI TOÁN VÀ NGHIỆP VỤ CƠ BẢN

1.1. Tổng quan bài toán

Hệ thống cung cấp nền tảng quản lý kho hàng, quan hệ khách hàng thông qua giao thức RESTful API. Mục tiêu cốt lõi là số hóa quy trình vận hành thương mại chăm sóc khách hàng, kiểm soát hàng hóa và xử lý giao dịch đơn hàng.

1.2. Mô hình thực thể cốt lõi

Thực thể	Vai trò trong hệ thống	Mối quan hệ nghiệp vụ
1. User	Tài khoản định danh truy cập hệ thống.	Phân quyền người dùng (ADMIN / USER), xác thực qua JWT.
2. Customer	Hồ sơ khách hàng mua sắm.	Lưu trữ thông tin liên hệ và sở hữu lịch sử nhiều đơn hàng.
3. Product	Danh mục hàng hóa thương mại.	Quản lý đơn giá và tồn kho ; tham chiếu trong chi tiết đơn hàng.
4. Order & Order Details	Giao dịch mua bán	Kết nối Customer và danh sách Product trong một giao dịch duy nhất.

1.3. Quy trình và Quy tắc nghiệp vụ cơ bản

1. Nghiệp vụ Xác thực & Phân quyền (Auth & Security):
    - Mật khẩu người dùng phải được mã hóa một chiều khi đăng ký.
    - Sử dụng cơ chế Stateless Authentication qua JWT Token cấp phát sau khi đăng nhập thành công. Các API quản trị (như xem danh sách tài khoản) yêu cầu quyền ADMIN.
2. Nghiệp vụ Quản lý khách hàng:
    - Kiểm soát tính duy nhất của Email và Số điện thoại trên toàn hệ thống khi tạo mới cũng như khi cập nhật hồ sơ.
    - Hỗ trợ tra cứu lịch sử đơn hàng theo từng khách hàng.
3. Nghiệp vụ Quản lý Tồn kho:
    - Bảo toàn lịch sử và Trạng thái kinh doanh: Sản phẩm không sử dụng cơ chế xóa khỏi CSDL.
    - Mỗi sản phẩm có trạng thái hoạt động (ACTIVE - Đang kinh doanh, INACTIVE - Ngừng kinh doanh). Chỉ các sản phẩm ACTIVE mới được phép giao dịch.

- Cô lập nghiệp vụ sửa thông tin và điều chỉnh kho: Việc cập nhật thông tin định danh (tên, giá bán, trạng thái) được tách rời hoàn toàn khỏi việc cập nhật số lượng tồn kho. Tồn kho chỉ biến động qua 3 luồng duy nhất:
    1. Khởi tạo sản phẩm ban đầu.
    2. Điều chỉnh kiểm kê/nhập hàng tăng giảm theo lượng chênh lệch khấu trừ tự động khi tạo đơn hàng và hoàn trả tự động khi hủy đơn.

4. Nghiệp vụ Xử lý Giao dịch Đơn hàng
    - Toàn vẹn giao dịch: Order và OrderDetails được xử lý trong một Transaction duy nhất.
    - Quy trình khởi tạo đơn và khấu trừ kho (POST /orders):
        1. Chuẩn hóa giỏ hàng: Kiểm tra đơn hàng không rỗng, số lượng mỗi sản phẩm lớn hơn 0. Tự động cộng dồn số lượng nếu có các productId trùng lặp trong cùng một body.
        2. Kiểm tra điều kiện ràng buộc: Toàn bộ sản phẩm trong đơn hàng phải đang ở trạng thái ACTIVE và có số lượng tồn kho hiện tại lớn hơn hoặc bằng số lượng đặt mua. Nếu bất kỳ sản phẩm nào vi phạm, hủy toàn bộ giao dịch.
        3. Ghi nhận và trừ kho: Khấu trừ tồn kho tương ứng, lưu đơn giá tại thời điểm mua vào order_details, tính tổng tiền và tạo đơn ở trạng thái PENDING.
    - Quy tắc Chuyển đổi trạng thái và hoàn kho:
        - Đơn hàng tuân thủ nghiêm ngặt luồng trạng thái một chiều (không cho phép lùi trạng thái):
            - Từ PENDING → chỉ được chuyển sang SHIPPING hoặc CANCELLED.
            - Từ SHIPPING → chỉ được chuyển sang COMPLETED hoặc CANCELLED.
            - Từ COMPLETED hoặc CANCELLED → Khóa hoàn toàn, chặn mọi thao tác thay đổi trạng thái.
        - Khi chuyển sang CANCELLED (từ PENDING hoặc SHIPPING), hệ thống thực thi Transaction: cập nhật trạng thái đơn sang CANCELLED và tự động cộng hoàn trả số lượng từng mặt hàng trong đơn về lại tồn kho của sản phẩm tương ứng (ngay cả khi sản phẩm đó hiện đã chuyển sang INACTIVE).

PHẦN 2: ĐẶC TẢ CÁC API (RESTful API SPECIFICATION)

- Base URL: http://localhost:3000/v3/api-docs
- Giao diện SwaggerUI : http://localhost:3000/swagger-ui/index.html

- Định dạng dữ liệu: json
- Cơ chế xác thực: JWT_Token (truyền qua Header Authorization đối với các endpoint yêu cầu đăng nhập).

2.1. Phân hệ Xác thực & Người dùng (Auth & Users)

Mục tiêu: Quản lý đăng ký, đăng nhập cấp phát Token và quản trị tài khoản hệ thống.

Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
POST	/auth/register	Đăng ký tài khoản mới	RegisterRequest (username, password)	201 Created - Message thành công	Kiểm tra trùng username; mã hóa password.
POST	/auth/login	Đăng nhập hệ thống	LoginRequest (username, password)	200 OK - Chuỗi JWT Token	Đối chiếu mật khẩu; sinh JWT nếu hợp lệ.
GET	/users	Lấy danh sách tài khoản	None	200 OK - List<UserDto>	Chỉ dành cho quyền ADMIN; ẩn trường mật khẩu.

2.2. Phân hệ Khách hàng (Customers)

Mục tiêu: Quản lý hồ sơ thông tin khách hàng và tra cứu lịch sử mua sắm.

Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
GET	/customers	Lấy danh sách khách hàng	Query: ?page=0&size=10	200 OK - Page<CustomerDto>	Truy vấn danh sách khách hàng có phân trang.
GET	/customers/{id}	Xem thông tin 1 khách hàng	Path: id	200 OK - CustomerDto	Xem chi tiết liên hệ và nhân viên Sales Rep phụ trách.
POST	/customers	Tạo mới hồ sơ khách hàng	CustomerRequest	201 Created - CustomerDto	Lưu thông tin khách hàng và gán nhân viên chăm sóc.
PUT	/customers/{id}	Cập nhật hồ sơ khách hàng	Path: id Body: CustomerRequest	200 OK - CustomerDto	Cập nhật thông tin hoặc đổi nhân viên chăm sóc.

Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
GET	/customers/{id}/orders	Xem lịch sử đơn hàng của KH	Path: id	200 OK - List<OrderResponseDto>	Liệt kê tất cả các đơn hàng khách hàng {id} đã đặt.

2.3. Phân hệ Sản phẩm (Products)

Mục tiêu: Quản lý danh mục hàng hóa, niêm yết giá và kiểm soát số lượng tồn kho.

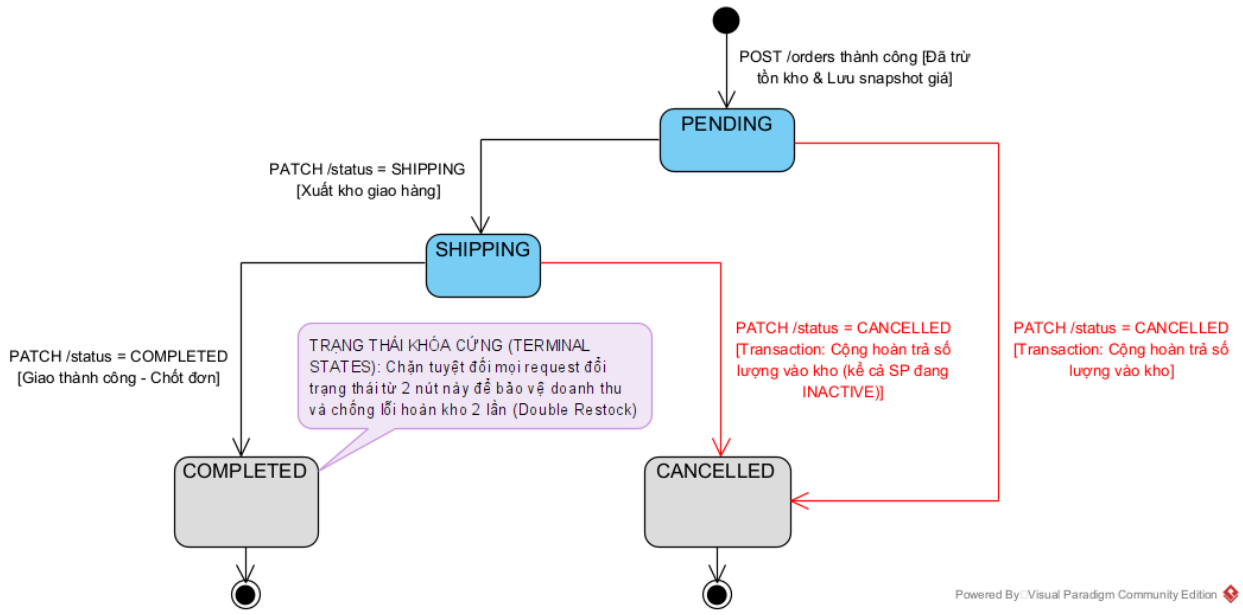
Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
GET	/products	Lấy danh sách sản phẩm	Query: ?page=0&size=10	200 OK - List<ProductDto>	Lọc theo tên sản phẩm, khoảng giá hoặc trạng thái kho.
GET	/products/{id}	Xem chi tiết sản phẩm	Path: id	200 OK - ProductDto	Truy xuất tên, đơn giá hiện tại và số lượng tồn kho.
POST	/products	Thêm mới sản phẩm và sửa số lượng	ProductRequest (name, qty, price)	201 Created - ProductDto	Ràng buộc số lượng và đơn giá. Xác thực người dùng admin.
PATCH	/products/{id}	Cập nhật thông tin thương mại	Path: id Body: ProductUpdateRequest (name, price, status)	200 OK - ProductDto	Cập nhật thông tin sản phẩm.

2.. Phân hệ Đơn hàng (Orders & Order Details)

Mục tiêu: Xử lý giao dịch mua bán trọn gói (gộp chung thông tin Order và Order Details).

Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
GET	/orders	Lấy danh sách đơn hàng	Query: ?page=0&size=1	200 OK - List<OrderSummaryDto>	Hiển thị thông tin tóm tắt các đơn hàng (không kèm list SP).

Method	Endpoint	Mô tả chức năng	Request Body / Params	Response	Quy tắc xử lý chính
GET	/orders/{id}	Xem chi tiết đơn hàng	Path: id	200 OK - OrderDetailResponse	Trả về thông tin chung kèm danh sách sản phẩm (order_details).
POST	/orders	Tạo đơn hàng mới	OrderRequest (customerId, employeeId, List<Items>)	201 Created - OrderDetailResponse	Chạy trong 1 Transaction: kiểm tra đủ tồn kho → tạo đơn & chi tiết đơn → trừ kho.
PATCH	/orders/{id}	Cập nhật trạng thái đơn hàng	Path: id Body: OrderRequest	200 OK - OrderDetailResponse	Cập nhập thái một chiều (PENDING → SHIPPING → COMPLETED). Nếu chuyển sang CANCELLED từ PENDING/SHIPPING: chạy Transaction đổi trạng thái và cộng hoàn kho theo order_details. Chặn tuyệt đối nếu đơn đã COMPLETED hoặc CANCELLED. Thay đổi số lượng sản phẩm khi đơn hàng ở trạng thái PENDING



PHẦN 3: MỘT SỐ VẤN ĐỀ CỦA HỆ THỐNG (ĐÃ XỬ LÝ Ở PHASE 1)

Khi Backend Server xử lý đồng thời nhiều HTTP Request (Concurrent Requests) và tương tác bất đồng bộ xuống Persistence Database, độ trễ đọc/ghi dễ gây sai lệch dữ liệu.

1. Lỗi ghi đè dữ liệu khi cập nhật tồn kho
   - Kịch bản gây lỗi:
     - Thực thể Product (mã A) đang có tồn kho qty = 10.
     - Lúc 09:00: Admin gọi GET /api/v1/products/A, nhận về Response có qty = 10 và chuẩn bị nhập thêm 5 sản phẩm (tính thành 15).
     - Lúc 09:01: Customer gửi Request POST /api/v1/orders với OrderRequest(customerId, employeeId, List<Items>) đặt mua 3 sản phẩm A. Server xử lý và trừ kho trong Database xuống còn qty = 7, trả về 201 Created.
     - Lúc 09:02: Admin gửi Request PUT mang toàn bộ dữ liệu cũ kèm ProductRequest(name, qty = 15, price) lên Server. Database bị ghi đè ngược thành qty = 15 (lệch 3 sản phẩm so với thực tế là $7 + 5 = 12$).
   - Cách hệ thống Phase 1 đã xử lý:
     - Trong vận hành thực tế, mọi biến động qty bắt buộc đi qua các Request giao dịch POST (như POST /api/v1/orders) hoặc điều chỉnh theo lượng chênh lệch .
2. Đứt gãy giao dịch giữa chừng
   - Kịch bản gây lỗi:

    - Client gửi POST /api/v1/orders với List<Items> gồm 5 sản phẩm. Server phải ghi liên tiếp vào bảng orders, order_details và trừ qty trong bảng products.
    - Khi Server vừa trừ kho xong 3 sản phẩm đầu trong Database, đến sản phẩm thứ 4 thì phát hiện qty không đủ (hoặc sản phẩm đang INACTIVE, hoặc ngắt kết nối Database đột ngột). Nếu các lệnh chạy độc lập, Server trả Response lỗi nhưng 3 sản phẩm đầu đã bị trừ kho vô lý trong khi đơn hàng không được tạo.
- Cách hệ thống Phase 1 đã xử lý:
    - Bọc toàn bộ quá trình kiểm tra trạng thái ACTIVE, đối chiếu và trừ/hoàn trả qty, lưu bảng orders + order_details (trong cả POST /api/v1/orders và PATCH /api/v1/orders/{id}) vào một Database Transaction (ACID) duy nhất.
    - Tuân thủ nguyên tắc All-or-Nothing: Bất kỳ bước nào lỗi sẽ tự động kích hoạt ROLLBACK, khôi phục nguyên trạng qty của các dòng trước đó.

3. Sai số trong Database khi có Request đồng thời
- Kịch bản gây lỗi:
    - Sản phẩm A còn tồn kho qty = 5. Cùng một thời điểm, 2 khách hàng gửi đồng thời 2 Request POST /api/v1/orders cùng đặt mua sản phẩm A với số lượng lần lượt là 4 và 3.
    - Hai luồng xử lý trên Server cùng đọc Database lúc qty = 5, cùng kiểm tra thấy hợp lệ ($5 \geq 4$ và $5 \geq 3$) và cùng phát lệnh cập nhật trừ kho xuống Database, dẫn đến bán vượt tồn kho (Overselling) hoặc ghi đè sai số dư qty.
- Cách hệ thống Phase 1 đã xử lý:
    - Áp dụng Khóa dòng mức cơ sở dữ liệu (Row-level Locking - SELECT ... FOR UPDATE) trên bảng products bên trong Transaction.
    - Khi Request 1 đang kiểm tra và trừ qty của sản phẩm A, dòng dữ liệu này bị khóa độc quyền. Request 2 phải chờ ở hàng đợi Database cho đến khi Request 1 COMMIT xong (lúc này qty còn 1), nhờ đó Request 2 đọc được giá trị mới nhất và báo lỗi hết hàng chính xác.

4. Bế tắc hệ thống do Khóa chéo (Database Deadlock)
- Kịch bản gây lỗi: Khi đã áp dụng khóa dòng (Row-level Locking) để chống âm kho cho POST /api/v1/orders:
    - Request 1 gửi List<Items> theo thứ tự [Sản phẩm A, Sản phẩm B] → Luồng 1 khóa dòng A trước, rồi chờ khóa dòng B.
    - Cùng lúc đó, Request 2 gửi List<Items> theo thứ tự [Sản phẩm B, Sản phẩm A] → Luồng 2 khóa dòng B trước, rồi chờ khóa dòng A.

    - Hệ quả: Luồng 1 chờ Luồng 2 nhả B, Luồng 2 chờ Luồng 1 nhả A → Cả hai treo cứng (Deadlock), làm nghẽn kết nối Database.
- Cách hệ thống Phase 1 đã phòng ngừa:
    - Ngay ở bước chuẩn hóa OrderRequest, hệ thống áp dụng quy tắc: "Gộp và sắp xếp productId trong List<Items> theo thứ tự tăng dần trước khi kiểm tra/trừ kho".
    - Nhờ đó, cả Request 1 và Request 2 đều luôn khóa dòng Sản phẩm A trước rồi mới đến Sản phẩm B, triệt tiêu hoàn toàn nguy cơ Deadlock.

PHẦN 4: NHỮNG ĐIỂM GỢI Ý CẢI TIẾN Ở PHASE 2

1. Request trùng lặp dữ liệu & Gửi lặp Request (Duplicate Request)
- Kịch bản gây lỗi:
    - Gửi lặp Request tạo đơn: Khi gọi POST /api/v1/orders/{id} Client gửi lặp 2 Request cùng lúc thay vì 1 request làm Server tạo thừa 1 đơn hàng và trừ bảng products 2 lần.

2. Lỗ hổng Thu hồi quyền truy cập của Stateless JWT (Token Revocation Problem)
- Kịch bản gây lỗi:
    - Ở Phase 1, API POST /api/v1/auth/login cấp phát JWT_Token theo cơ chế Stateless (Server không lưu trạng thái đăng nhập mà chỉ kiểm tra chữ ký và hạn sử dụng của Token trên Header Authorization).
    - Nếu một tài khoản User (quyền USER) bị lộ Token hoặc nhân viên bị cho nghỉ việc đột xuất khi Token vẫn còn hạn (ví dụ: 24 giờ), người này vẫn có thể dùng Token cũ gọi GET /api/v1/customers xem dữ liệu khách hàng hoặc gửi Request phá hoại hệ thống cho đến khi Token tự hết hạn.
- Hướng giải quyết (Kiến nghị nâng cấp):
    - Giảm thời gian sống của Access Token xuống ngắn ($15 - 30$ phút) kết hợp với Refresh Token.
    - Tích hợp bộ nhớ đệm (Redis Cache) ở Phase 2 để lưu danh sách đen (Token Blacklist) hoặc kiểm tra tức thời trạng thái hoạt động của nhân viên (ACTIVE / TERMINATED) trước khi cho phép thực thi Request.

3. Mở rộng xử lý nghiệp vụ và Tối ưu hóa hiệu năng
- Mở rộng thực thể tham gia vào hệ thống (Employee):
    - Bổ sung thực thể Employee với mối quan hệ tự tham chiếu đệ quy (reportsTo) để quản lý sơ đồ phân cấp báo cáo.

    - Chuẩn hóa liên kết giữa Employee với Customer (nhân viên Sales Rep chăm sóc trong POST/PUT /api/v1/customers) và với Order (trường employeeId trong POST /api/v1/orders) nhằm thống kê KPI doanh số.
- Phân quyền chi tiết & Tối ưu hóa truy vấn:
    - Liên kết tài khoản User với hồ sơ Employee để nâng cấp phân quyền từ mức ADMIN/USER sang phân quyền truy cập dữ liệu theo cấp bậc.
    - Mở rộng bộ lọc tìm kiếm đa tiêu chí trên GET /api/v1/products và GET /api/v1/orders (lọc theo khoảng giá price, ngưỡng tồn kho qty, thời gian đặt hàng) kết hợp phân trang (?page=0&size=10).
    - Tích hợp Caching (Redis) cho danh mục sản phẩm (GET /api/v1/products) để giảm tải đọc trực tiếp xuống Persistence Database.
- Bổ sung bảng lịch sử biến động kho (Inventory Audit Log):
    - Ghi nhận tự động mọi thay đổi về qty của Product (khi tạo sản phẩm POST /api/v1/products, điều chỉnh kho PATCH /api/v1/products/{id}, trừ kho khi POST /api/v1/orders hoặc hoàn kho khi PATCH /api/v1/orders/{id} sang CANCELLED), lưu rõ employeeId/User thực hiện và lý do điều chỉnh.
        -